

Contents

Activity 1 – Installing and Verifying Podman on Ubuntu WSL	2
Activity 2 – Exploring the Podman Environment	5
Activity 3 – Search and Pull Images from a Registry	7
Activity 4 – Understanding Image Layers	13
Activity 5 – Working with Detached Containers	24
Activity 6 – Executing Commands Inside Running Containers	30
Activity 7 – Bind Mounts	36
Activity 8 – Named Volumes	43
Activity 9 – Create a Custom Network	50
Activity 10 – Creating Your First Pod	55
Activity 11 – Deploy Multiple Containers Inside a Pod	57
Activity 12 – Managing Pods	58
Activity 13 – Build Your First Image Using a Dockerfile	59
Activity 14 – Build a Multi-stage Image Using a Dockerfile	68
Activity 15 – Build Images with Multiple Tags	78
Activity 16 – Build Images Without Using the Build Cache	82

Activity 1 – Installing and Verifying Podman on Ubuntu WSL

Estimated Time: 15 Minutes

Objective

In this activity, you will install Podman on Ubuntu WSL and verify that it is working correctly.

Learning Outcomes

After completing this activity, you will be able to:

- Install Podman
- Verify the installation
- Check the installed version
- Display Podman system information

Prerequisites

- Windows 11
- Ubuntu WSL installed
- Internet connectivity

Step 1: Update Ubuntu

Run the following command:

```
sudo apt update
```

Explanation

Updates the package repository so Ubuntu knows about the latest available software.

Step 2: Upgrade Existing Packages

```
sudo apt upgrade -y
```

Explanation

Upgrades installed packages to their latest versions.

Step 3: Install Podman

```
sudo apt install -y podman
```

Explanation

Downloads and installs Podman along with all required dependencies.

Step 4: Verify Installation

```
podman --version
```

Expected Output

```
podman version 5.x.x
```

(The version may differ depending on the Ubuntu repository.)

Step 5: Display Podman Information

podman info

Explanation

Displays:

- Storage Driver
- OCI Runtime
- Image Store
- Rootless Configuration
- Cgroup Information
- Network Backend

Step 6: Verify the OCI Runtime

Run:

```
podman info --format "{{.Host.OCIRuntime.Name}}"
```

Expected Output

crun

or

runc

crun is a lightweight, high-performance OCI (Open Container Initiative) runtime used to create and run containers. It is an alternative to runc, which is the default OCI runtime used by Docker and many Kubernetes distributions.

In simple terms:

- *Docker or Podman receives your command (for example, docker run nginx or podman run nginx).*
- *The container engine prepares the container image and configuration.*
- *It then hands the work over to an OCI runtime such as crun or runc.*
- *The runtime interacts with the Linux kernel to create namespaces, cgroups, mount the filesystem, and start the container process.*

Step 7: Verify Storage Location

```
podman info --format "{{.Store.GraphRoot}}"
```

Example Output

/home/mausomi/.local/share/containers/storage

Verification

Run:

```
podman info
```

Knowledge Check

1. Which command displays the installed Podman version?
2. Which command displays detailed Podman configuration?

3. Which OCI runtime is being used?

Cleanup

No cleanup is required.

Summary

You successfully:

- Installed Podman
- Verified the installation
- Examined the Podman configuration
- Verified the OCI runtime and storage configuration

Activity 2 – Exploring the Podman Environment

Estimated Time: 20 Minutes

Objective

Learn how to inspect the local Podman environment before creating containers.

Step 1: Display System Information

podman system info

Observe:

- Host
- Store
- Registries
- Network Backend
- Version

Step 2: View Available Registries

cat /etc/containers/registries.conf

Question:

Which registry appears first?

Step 3: List Existing Images

podman images

Expected:

REPOSITORY TAG IMAGE ID CREATED SIZE

If no images exist, the list will be empty.

Step 4: List Existing Containers

podman ps -a

Expected:

No containers, or previously created containers.

Step 5: Check Available Networks

podman network ls

Step 6: Check Available Volumes

podman volume ls

Step 7: Display Available Podman Commands

podman --help

Try to identify commands related to:

- Images
- Containers
- Volumes
- Networks

Challenge

Without using Google,
find the command that displays:

- running containers
- local images
- local volumes

using only
`podman --help`

Activity 3 – Search and Pull Images from a Registry

Objective

Learn how to search for container images available in a registry, understand image tags, pull specific image versions, and verify downloaded images in the local Podman environment. Participants will learn how container images are discovered, downloaded, and managed before creating containers.

Learning Outcomes

After completing this activity, participants will be able to:

- Understand the role of container registries.
- Search for images using Podman.
- Identify official and community images.
- Understand image tags and versions.
- Pull a specific image version instead of using the default latest tag.
- Verify downloaded images locally.
- Inspect image details.

Estimated Time

20 minutes

Prerequisites

Before starting this activity:

1. Podman must be installed

Verify:

```
podman --version
```

Example:

```
podman version 5.4.0
```

2. Internet connectivity

Podman requires internet access to communicate with container registries.

3. Basic understanding of containers

Participants should know:

- Containers run from images.
- Images are templates used to create containers.

What is a Container Image?

A container image is a read-only package containing everything required to run an application:

- Application code
- Runtime libraries
- System tools
- Configuration files
- Dependencies

Example:

An Ubuntu image contains:

Ubuntu OS Files

+

Libraries

+

Utilities

A container is created from this image.

Image

|

|

v

Container

What is a Container Registry?

A registry is a location where container images are stored and distributed.

Common registries:

- Docker Hub
- Red Hat Quay
- Oracle Container Registry
- GitHub Container Registry

Example:

Registry

|

|

+---- ubuntu

|

+---- nginx

|

+---- mysql

Image Naming Convention

A container image usually follows this format:

registry/repository/image:tag

Example:

docker.io/library/ubuntu:22.04

Breaking it down:

Component	Meaning
docker.io	Registry
library	Official image repository
ubuntu	Image name
22.04	Image tag

Step 1: Search for an Ubuntu Image

Run:

```
podman search ubuntu
```

Command Explanation

Command	Explanation
podman	Podman CLI
search	Search images from registries
ubuntu	Image keyword

Expected Result

Example:

NAME	DESCRIPTION
docker.io/library/ubuntu	Ubuntu Linux
docker.io/ubuntu	Ubuntu Official Image
quay.io/ubuntu	Ubuntu Image

Step 2: Understand Search Results

Example:

```
docker.io/library/ubuntu
```

means:

Registry

|

v

docker.io

Repository

|

v

library

Image

|
v
ubuntu

Step 3: Search for Nginx Image

Run:

```
podman search nginx
```

Expected:

NAME	DESCRIPTION
docker.io/library/nginx	Official nginx image

Step 4: Explore Image Tags

An image can have multiple versions.

Example:

```
ubuntu:latest
```

```
ubuntu:22.04
```

```
ubuntu:20.04
```

```
ubuntu:24.04
```

The tag identifies the version.

Why Avoid Always Using latest?

Example:

Today:

```
ubuntu:latest → Ubuntu 24.04
```

After some months:

```
ubuntu:latest → Ubuntu 26.04
```

Your application may behave differently.

For production environments, use fixed versions.

Example:

```
ubuntu:22.04
```

instead of:

```
ubuntu:latest
```

Step 5: Pull the Latest Ubuntu Image

Run:

```
podman pull ubuntu
```

Command Explanation

Command	Explanation
pull	Downloads image

ubuntu	Image name
--------	------------

Expected Result

Example:

Trying to pull docker.io/library/ubuntu:latest...

Getting image source signatures

Copying blob ...

Copying config ...

Writing manifest ...

Storing signatures

Step 6: Pull a Specific Version

Instead of latest:

podman pull ubuntu:22.04

Command Explanation

ubuntu:22.04

contains:

- Image name → ubuntu
- Tag → 22.04

Expected:

Trying to pull docker.io/library/ubuntu:22.04

Successfully pulled image

Step 7: Verify Downloaded Images

Run:

podman images

Command Explanation

Displays locally available images.

Expected Output:

REPOSITORY	TAG	IMAGE ID
docker.io/library/ubuntu	latest	35a888...
docker.io/library/ubuntu	22.04	8feb4...

Step 8: Display Detailed Image Information

Run:

```
podman inspect ubuntu:22.04
```

Command Explanation

inspect displays metadata:

- Image ID
- Creation date
- Architecture
- Environment variables
- Layers

Example:

```
"Architecture": "amd64"
```

```
"Os": "linux"
```

```
"Created":
```

```
"2025-05-01"
```

Step 9: View Image Size

Run:

```
podman images --format "{{.Repository}} {{.Tag}} {{.Size}}"
```

Example:

```
ubuntu latest 78MB
```

```
ubuntu 22.04 77MB
```

Expected Results

After completing this activity, participants should have:

Local Images:

```
ubuntu:latest
```

```
ubuntu:22.04
```

Example:

REPOSITORY	TAG
------------	-----

docker.io/library/ubuntu	latest
--------------------------	--------

docker.io/library/ubuntu	22.04
--------------------------	-------

Activity 4 – Understanding Image Layers

Objective

Explore how container images are internally built using multiple layers and understand how image layers are reused to optimize storage and improve download performance.

Participants will learn how images share common layers and how changes create new layers.

Learning Outcomes

After completing this activity, participants will be able to:

- Understand the concept of container image layers.
- Identify individual layers inside an image.
- View image history.
- Compare related images.
- Understand layer sharing between images.
- Understand how image layering reduces storage usage.
- Recognize the relationship between Dockerfile instructions and image layers.

Estimated Time

25 minutes

Prerequisites

Before starting this activity:

1. Podman installed

Verify:

```
podman --version
```

Example:

```
podman version 5.4.0
```

2. Images downloaded

The following images will be used:

- Ubuntu
- Ubuntu related image

Example:

```
podman pull ubuntu:22.04
```

3. Basic understanding of container images

Participants should understand:

Image → Container

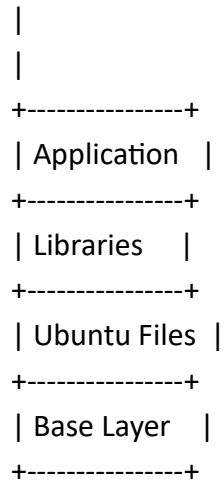
An image is a template used to create containers.

What are Image Layers?

A container image is not stored as one single file.
Instead, it is built using multiple read-only layers.

Example:

Container Image



Each layer represents a change made during image creation.

Why Use Layers?

Layers provide:

1. Reusability

Multiple images can share the same layers.

Example:

ubuntu image

Layer 1: Ubuntu OS

Layer 2: Libraries

nginx image

Layer 1: Ubuntu OS

Layer 2: Libraries

Layer 3: nginx

The first two layers are reused.

2. Faster Downloads

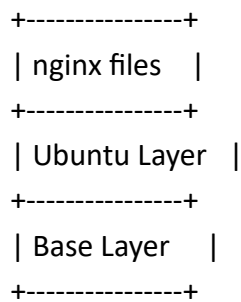
If a layer already exists locally, Podman does not download it again.

3. Storage Optimization

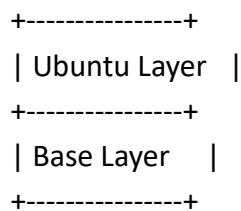
Shared layers consume storage only once.

Image Layer Architecture

nginx Image



Ubuntu Image



The common layers are shared.

Step 1: Pull Ubuntu Image

Run:

```
podman pull ubuntu:22.04
```

Command Explanation

Command	Explanation
pull	Downloads image
ubuntu:22.04	Ubuntu version 22.04

Expected:

Successfully pulled ubuntu:22.04

Step 2: Pull Ubuntu-Based Nginx Image

Run:

```
podman pull nginx
```

Command Explanation

This downloads the Nginx image.

Nginx image contains:

- Base operating system files
- Required libraries

- Nginx application

Expected:

Successfully pulled docker.io/library/nginx

Step 3: List Downloaded Images

Run:

podman images

Example:

REPOSITORY	TAG
ubuntu	22.04
nginx	latest

Step 4: Compare Image Sizes

Run:

podman images

Example:

REPOSITORY	TAG	SIZE
ubuntu	22.04	77 MB
nginx	latest	190 MB

Observation

Nginx is larger because it contains:

Ubuntu Base

+

Libraries

+

Nginx Software

+

Configuration

Step 5: View Image History

Run:

podman history ubuntu:22.04

Command Explanation

history displays:

- Image layers

- Creation time
- Commands used during image creation
- Layer size

Example:

IMAGE ID	CREATED	SIZE
abc123	2 weeks ago	77MB
<missing>	2 weeks ago	30MB
<missing>	2 weeks ago	45MB

Step 6: View Nginx Image History

Run:

```
podman history nginx
```

Example:

IMAGE ID	CREATED	SIZE
layer1	Base OS	80MB
layer2	Install nginx	50MB
layer3	Config files	5MB

Step 7: Inspect Image Layers

Run:

```
podman inspect ubuntu:22.04
```

Look for:

RootFS

|

Layers

Example:

```
"Layers": [
  "sha256:abc123",
  "sha256:def456"
]
```

Step 8: Compare Layer IDs

Run:

```
podman inspect ubuntu:22.04 | grep sha256
and:
```

```
podman inspect nginx | grep sha256
```

Observation

Some layer IDs may appear in both images.

Example:

Ubuntu:

sha256:111aaa

sha256:222bbb

Nginx:

sha256:111aaa

sha256:222bbb

sha256:333ccc

Common layers:

sha256:111aaa

sha256:222bbb

Expected Results

After completing this activity, participants should understand:

Ubuntu Image

Base Layer

+

Ubuntu Files

Nginx Image

Base Layer

+

Ubuntu Files

+

Nginx Application

Shared layers are stored only once.

Verification

Verify the following:

Task	Command	Expected Result
Images downloaded	podman images	Images displayed
View layers	podman history image	Layer information
Inspect image	podman inspect image	Layer IDs displayed
Compare images	podman images	Different sizes visible

CONTAINER LIFECYCLE

Step 1: Create a Container

Create a container from Nginx image.

```
podman create --name lifecycle-demo docker.io/library/nginx:latest
```

Command Explanation

Command	Meaning
create	Creates a container
--name	Assigns container name
lifecycle-demo	Container name
nginx	Image

Expected:

<container-id>

Example:

a72bd9f8c21e

Step 2: Check Container Status

Run:

```
podman ps -a
```

Expected:

```
CONTAINER ID  IMAGE  STATUS  NAME
```

```
a72bd9f      nginx  Created  lifecycle-demo
```

Observation

The container exists but is not running.

Current state:

Created

Step 3: Start the Container

Run:

```
podman start lifecycle-demo
```

Expected:

```
lifecycle-demo
```

Check:

```
podman ps
```

Expected:

CONTAINER ID	STATUS	NAME
--------------	--------	------

a72bd9f	Up 5 seconds	lifecycle-demo
---------	--------------	----------------

Current state:

Created

|

v

Running

Step 4: View Running Processes

Run:

```
podman top lifecycle-demo
```

Command Explanation

podman top displays processes running inside a container.

Example:

USER	PID	COMMAND
------	-----	---------

root	1	nginx
------	---	-------

Step 5: Pause the Container

Run:

```
podman pause lifecycle-demo
```

What happens?

Podman freezes the processes inside the container.

The container:

- Still exists
- Keeps memory allocated
- Stops executing processes temporarily

Check status:

```
podman ps -a
```

Expected:

STATUS

Up (Paused)

Lifecycle:

Running

|

v

Paused

Step 6: Resume the Container

Run:

```
podman unpause lifecycle-demo
```

Command Explanation

unpause resumes previously paused processes.

Verify:

```
podman ps
```

Expected:

STATUS

Up

Lifecycle:

Paused

|

v

Running

Step 7: Restart the Container

Run:

```
podman restart lifecycle-demo
```

What happens internally?

Restart performs:

Stop container

|

v

Start container

Verify:

```
podman ps
```

Expected:

STATUS

Up

Step 8: Stop the Container

Run:

```
podman stop lifecycle-demo
```

Command Explanation

stop gracefully terminates the container.

Podman sends:

SIGTERM

Then waits before forcing termination.

Check:

```
podman ps -a
```

Expected:

STATUS

Exited

Lifecycle:

Running

|

v

Stopped

Step 9: Start a Stopped Container Again

Run:

```
podman start lifecycle-demo
```

Verify:

```
podman ps
```

Expected:

STATUS

Up

Step 10: Remove the Container

First stop it:

```
podman stop lifecycle-demo
```

Remove:

```
podman rm lifecycle-demo
```

Command Explanation

Command Meaning

rm	Removes container
----	-------------------

Verify:

```
podman ps -a
```

Expected:

No containers found

Expected Results

After completing this activity:

The participant should have performed:

Create

|

v

Start

|

v

Running

|

v

Pause

|

v

Resume

|

v

Restart

|

v

Stop

|

v

Remove

Activity 5 – Working with Detached Containers

Objective

Learn how to run containers in **detached mode**, monitor their execution, view application logs, and verify that the application is accessible from a web browser.

Participants will understand the difference between foreground and detached containers and learn how detached mode is commonly used for production workloads.

Learning Outcomes

After completing this activity, participants will be able to:

- Understand foreground and detached container execution.
- Deploy an Nginx web server in detached mode.
- Verify running containers.
- View container logs.
- Access the web application from a browser.
- Stop and remove detached containers.

Estimated Time

25–30 minutes

Prerequisites

Before starting this activity, ensure:

1. Podman is installed

Verify:

```
podman --version
```

2. Nginx image is available

Check:

```
podman images
```

If not available:

```
podman pull docker.io/library/nginx:latest
```

3. Verify no process is using port 8080

Linux:

```
ss -tuln | grep 8080
```

Expected:

No output.

What is Detached Mode?

By default, containers run in the **foreground**.

Example:

```
podman run docker.io/library/nginx
```

The terminal remains attached to the container.

Terminal

|



Container

You cannot use the same terminal until the container stops or you detach.

Detached Mode

Detached mode starts the container **in the background**.

Example:

```
podman run -d docker.io/library/nginx
```

The terminal immediately returns.

Terminal

|



Container (Running in Background)

You can continue executing other commands.

Why Use Detached Mode?

Detached mode is commonly used for:

- Web servers
- Databases
- APIs
- Monitoring tools
- Production services

Most production containers run in detached mode because administrators do not need to keep a terminal attached.

Step-by-Step Instructions

Step 1: Run an Nginx Container in Detached Mode

Execute:

```
podman run -d \  
--name nginx-detached \  
-p 8080:80 \  
docker.io/library/nginx:latest
```

Command Explanation

Option	Description
run	Creates and starts a container
-d	Runs the container in detached (background) mode
--name nginx-detached	Assigns a meaningful name
-p 8080:80	Maps host port 8080 to container port 80
docker.io/library/nginx:latest	Nginx image

Understanding Port Mapping

Host Machine

Port 8080



Container

Port 80 (Nginx)

Requests sent to **localhost:8080** are forwarded to the Nginx server inside the container.

Expected Result

Podman returns only the container ID.

Example:

ab34cd56ef7890...

This indicates the container is running in the background.

Step 2: Verify the Running Container

Run:

podman ps

Command Explanation

podman ps displays only running containers.

Expected Output

CONTAINER ID	IMAGE	STATUS	PORTS	NAMES
--------------	-------	--------	-------	-------

ab34cd56ef78	nginx	Up	0.0.0.0:8080->80/tcp	nginx-detached
--------------	-------	----	----------------------	----------------

Observe:

- Container is running.
- Port mapping is displayed.
- Container name is shown.

Step 3: View Container Logs

Run:

```
podman logs nginx-detached
```

Command Explanation

Displays everything written to:

- Standard Output (stdout)
- Standard Error (stderr)

Applications typically write diagnostic information here.

Expected Output

Initially, there may be little or no output because Nginx does not log much until it receives requests.

Step 4: Access the Web Page

Open a web browser.

Navigate to:

```
http://localhost:8080
```

Expected Result

The default Nginx welcome page should appear.

Example:

Welcome to nginx!

If you see this page,
the nginx web server is successfully installed...

Step 5: Generate Log Entries

Refresh the browser several times.

or

Use curl:

```
curl http://localhost:8080
```

Now view the logs again:

```
podman logs nginx-detached
```

Expected Output

```
GET / HTTP/1.1
```

```
GET /favicon.ico HTTP/1.1
```

Depending on the Nginx configuration, access logs may be visible.

Step 6: Follow Live Logs

Run:

```
podman logs -f nginx-detached
```

Command Explanation

Option	Description
-f	Follow logs continuously

This behaves similarly to:

```
tail -f logfile
```

Leave this command running.

Refresh the browser.

New log entries should appear immediately.

Exit log monitoring:

Ctrl + C

Step 7: Inspect the Container

Run:

```
podman inspect nginx-detached
```

Look for:

- Container ID
- Image
- Network
- Port mappings
- State

Step 8: Stop the Container

Run:

```
podman stop nginx-detached
```

Expected Output

```
nginx-detached
```

Verify:

```
podman ps
```

Expected:

No running containers

Step 9: Remove the Container

Run:

```
podman rm nginx-detached
```

Verify:

```
podman ps -a
```

The container should no longer exist.

Expected Results

At the end of the activity:

- Nginx was deployed in detached mode.
- The web page was accessible on port **8080**.
- Logs were viewed.
- Live log monitoring was demonstrated.
- The container was stopped and removed.

Lifecycle:

Run (-d)



Running



View Logs



Access Web Page



Stop



Remove

Verification

Task	Command	Expected Result
Run detached	podman run -d	Container starts
View running containers	podman ps	Container displayed
View logs	podman logs	Logs displayed
Follow logs	podman logs -f	Live logs displayed
Browser test	http://localhost:8080	Nginx welcome page
Stop container	podman stop	Container stopped
Remove container	podman rm	Container removed

Activity 6 – Executing Commands Inside Running Containers

Objective

Learn how to execute commands inside a running container using **podman exec**, access the container shell, install software packages, verify the installation, and safely exit the container.

This activity helps participants understand the difference between interacting with the host operating system and working inside a running container.

Learning Outcomes

After completing this activity, participants will be able to:

- Access a running container interactively.
- Execute commands inside a container.
- Understand the difference between podman exec and podman run.
- Install software inside a running container.
- Verify package installation.
- Exit the container safely.
- Understand that changes made inside a container are temporary unless committed to a new image or stored in a volume.

Estimated Time

25–30 minutes

Prerequisites

- Podman must be installed.

Verify:

```
podman --version
```

- The Nginx image should be available locally.

Check:

```
podman images
```

If not available:

```
podman pull docker.io/library/nginx:latest
```

- Internet connectivity is required to install packages inside the container.

- A running container is required.

Start one if necessary:

```
podman run -d \  
--name nginx-shell \  
-p 8080:80 \  
docker.io/library/nginx:latest
```

Verify:

```
podman ps
```

Containers are isolated environments. Sometimes administrators need to access a running container to troubleshoot problems, inspect files, install diagnostic tools, verify application configurations, or monitor processes.

The podman exec command allows you to execute commands inside an already running container without creating a new one.

General syntax:

```
podman exec [OPTIONS] CONTAINER COMMAND
```

Example:

```
podman exec nginx-shell ls /
```

This executes the `ls /` command inside the running container and immediately returns the output.

For an interactive shell, use:

```
podman exec -it nginx-shell bash
```

If the image does not include Bash, use:

```
podman exec -it nginx-shell sh
```

The options used are:

Option Description

- | | |
|----|--|
| -i | Keeps standard input open (interactive mode) |
| -t | Allocates a terminal (TTY) |

Step-by-Step Instructions

Step 1: Verify the Running Container

List all running containers:

```
podman ps
```

Expected output:

CONTAINER ID	IMAGE	STATUS	NAMES
--------------	-------	--------	-------

abcd1234	nginx	Up	nginx-shell
----------	-------	----	-------------

Step 2: Execute a Simple Command

Display the hostname of the container:

```
podman exec nginx-shell hostname
```

Expected output:

```
abcd1234ef56
```

The hostname is unique to the container and is different from the host machine.

Step 3: Display the Current User

Execute:

```
podman exec nginx-shell whoami
```

Expected output:

```
root
```

Most official container images execute commands as the **root** user unless configured otherwise.

Step 4: Open an Interactive Shell

Start a shell session inside the running container:

```
podman exec -it nginx-shell bash
```

If Bash is unavailable:

```
podman exec -it nginx-shell sh
```

Expected prompt:

```
root@abcd1234:/#
```

You are now working inside the container.

Step 5: Verify the Operating System

Display operating system information:

```
cat /etc/os-release
```

Example output:

```
PRETTY_NAME="Debian GNU/Linux 12"
```

This confirms the Linux distribution used by the container image.

Step 6: Update the Package Repository

Update the package index:

```
apt update
```

This downloads the latest package information from the configured repositories.

Note: If you are using an image based on Alpine Linux, use apk instead of apt. If you are using Oracle Linux, RHEL, or Fedora, use dnf.

Step 7: Install a Package

Install **curl**:

```
apt install -y curl
```

The -y option automatically accepts the installation prompts.

Step 8: Verify the Installation

Check the installed version:

```
curl --version
```

Example output:

```
curl 8.x.x
```

This confirms that the package has been installed successfully.

Step 9: Locate the Installed Binary

Execute:

```
which curl
```

Expected output:

```
/usr/bin/curl
```

This displays the location of the installed executable.

Step 10: Exit the Container

Leave the interactive shell:

```
exit
```

You should return to your host terminal.

Example:

```
mausomi@LAPTOP:~$
```

Step 11: Verify the Container is Still Running

Although you exited the shell, the container continues to run.

Verify:

```
podman ps
```

Expected output:

CONTAINER ID	IMAGE	STATUS	NAMES
--------------	-------	--------	-------

abcd1234	nginx	Up	nginx-shell
----------	-------	----	-------------

Step 12: Stop and Remove the Container

Stop the container:

```
podman stop nginx-shell
```

Remove the container:

```
podman rm nginx-shell
```

Command Explanations

Command	Description
podman exec	Executes a command inside a running container
podman exec -it	Opens an interactive terminal inside a running container
hostname	Displays the container's hostname
whoami	Displays the current user
cat /etc/os-release	Displays operating system information
apt update	Updates package repository information

apt install	Installs software packages
which	Displays the location of an executable
exit	Closes the interactive shell session
podman stop	Stops a running container
podman rm	Removes a stopped container

Expected Results

At the end of this activity, you should be able to:

- Access a running container using podman exec.
- Execute commands without opening a shell.
- Open an interactive shell inside the container.
- Install a package successfully.
- Verify the installed package.
- Exit the container while keeping it running.
- Stop and remove the container when finished.

Activity 7 – Bind Mounts

Objective

Learn how to use **bind mounts** in Podman to share files and directories between the host system and a container. Participants will mount a local HTML directory into an Nginx container and observe how changes made on the host are immediately reflected inside the container.

Learning Outcomes

After completing this activity, participants will be able to:

- Understand the concept of bind mounts.
- Mount a host directory into a container.
- Verify mounted files inside the container.
- Modify files on the host and observe changes inside the container.
- Understand common use cases for bind mounts during development.

Estimated Time

25–30 minutes

Prerequisites

- Podman must be installed.

Verify:

```
podman --version
```

- The Nginx image should be available locally.

Verify:

```
podman images
```

If not available:

```
podman pull docker.io/library/nginx:latest
```

- Create a directory on the host that will be mounted into the container.

Example:

```
mkdir ~/website
```

Navigate to the directory:

```
cd ~/website
```

Create a simple HTML page:

```
cat > index.html <<EOF
<!DOCTYPE html>
<html>
<head>
  <title>Podman Bind Mount Demo</title>
</head>
<body>
  <h1>Welcome to Podman!</h1>
  <p>This page is served from a bind mount.</p>
</body>
</html>
EOF
```

Background

A **bind mount** allows a container to access a directory or file that already exists on the host system.

Unlike copying files into a container, a bind mount creates a direct connection between the host directory and the container.

Host Machine

~/website

|

|



Container

/usr/share/nginx/html

Any changes made in the host directory are immediately visible inside the container.

Bind mounts are commonly used for:

- Web application development
- Configuration files
- Source code editing
- Log file access
- Testing applications without rebuilding images

The general syntax is:

```
podman run -v HOST_DIRECTORY:CONTAINER_DIRECTORY IMAGE
```

Step-by-Step Instructions

Step 1: Verify the HTML File

Display the contents of the file:

```
cat ~/website/index.html
```

Verify that the HTML file exists before starting the container.

Step 2: Start an Nginx Container with a Bind Mount

Run the container:

```
podman run -d \  
  --name bind-demo \  
  -p 8080:80 \  
  -v ~/website:/usr/share/nginx/html:Z \  
  docker.io/library/nginx:latest
```

Note: The :Z option is recommended on SELinux-enabled systems such as Fedora, RHEL, Oracle Linux, and CentOS. If you are using Ubuntu or WSL without SELinux, the mount usually works without :Z.

Command Explanation

Option	Description
-d	Run the container in detached mode
--name	Assign a container name
-p 8080:80	Publish container port 80 on host port 8080

Option	Description
-v	Create a bind mount
~/website	Directory on the host
/usr/share/nginx/html	Web root inside the container
:Z	Relabel files for SELinux access

Step 3: Verify the Running Container

Execute:

```
podman ps
```

Verify that the container is running.

Step 4: Access the Website

Open a browser:

<http://localhost:8080>

You should see:

Welcome to Podman!

This page is served from a bind mount.

Alternatively:

```
curl http://localhost:8080
```

Step 5: Verify the Mounted Files Inside the Container

Display the contents of the mounted directory:

```
podman exec bind-demo ls /usr/share/nginx/html
```

Expected output:

index.html

Display the file contents:

```
podman exec bind-demo cat /usr/share/nginx/html/index.html
```

The content should match the host file.

Step 6: Modify the HTML File on the Host

Open the file using your preferred editor.

Example:

```
nano ~/website/index.html
```

Change the content to:

```
<!DOCTYPE html>

<html>

<head>

  <title>Updated Page</title>

</head>

<body>

  <h1>Podman Bind Mount Successful!</h1>

  <p>This page was updated from the host machine.</p>

</body>

</html>
```

Save the file.

Step 7: Refresh the Browser

Reload:

```
http://localhost:8080
```

The updated page should appear immediately.

No container restart is required.

Step 8: Verify the Updated File Inside the Container

Run:

```
podman exec bind-demo cat /usr/share/nginx/html/index.html
```

The updated content should be displayed.

Step 9: Inspect the Container

Inspect the container configuration:

```
podman inspect bind-demo
```

Locate the **Mounts** section.

Example:

```
"Mounts": [  
  {  
    "Destination": "/usr/share/nginx/html",  
    "Source": "/home/mausomi/website",  
    "Type": "bind"  
  }  
]
```

This confirms that the directory is mounted from the host.

Step 10: Stop and Remove the Container

Stop the container:

```
podman stop bind-demo
```

Remove it:

```
podman rm bind-demo
```

Verify:

```
podman ps -a
```

The container should no longer exist.

The **website directory on the host remains unchanged.**

Command Explanations

Command	Description
podman run -v	Creates a bind mount
podman exec	Executes commands inside the container
ls	Lists files in a directory
cat	Displays file contents
podman inspect	Displays detailed container configuration
podman stop	Stops a running container
podman rm	Removes a stopped container

Expected Results

After completing this activity, participants should be able to:

- Create a bind mount.
- Access host files from inside the container.
- Modify files on the host.
- Observe changes immediately in the running container.
- Verify bind mount configuration using `podman inspect`.

Activity 8 – Named Volumes

Objective

Learn how to create and use **named volumes** in Podman to store persistent data. Participants will create a Podman-managed volume, mount it into a container, store data, remove the container, and verify that the data persists.

Learning Outcomes

After completing this activity, participants will be able to:

- Create a named volume.
- Mount a named volume into a container.
- Store persistent data inside the volume.
- Verify that data remains available after the container is removed.
- Inspect and manage Podman volumes.

Estimated Time

25–30 minutes

Prerequisites

- Podman must be installed.

Verify:

`podman --version`

- Verify existing volumes:

`podman volume ls`

- The Ubuntu image should be available.

`podman images`

If not available:

`podman pull docker.io/library/ubuntu:22.04`

Unlike **bind mounts**, where the data resides in a directory on the host, **named volumes** are created and managed by Podman.

A named volume is stored in Podman's storage location and can be attached to one or more containers.

Benefits of named volumes include:

- Persistent application data
- Easier backup and migration
- Better portability
- Isolation from the host filesystem
- Data survives container deletion

General syntax:

```
podman volume create VOLUME_NAME
```

Mounting a named volume:

```
podman run -v VOLUME_NAME:/container/path IMAGE
```

Unlike bind mounts, you do not specify a host directory.

Step-by-Step Instructions

Step 1: List Existing Volumes

Run:

```
podman volume ls
```

Initially, you may see no volumes.

Example:

DRIVER	VOLUME NAME
--------	-------------

Step 2: Create a Named Volume

Create a volume named **website-data**.

```
podman volume create website-data
```

Expected output:

```
website-data
```

Step 3: Verify the Volume

List all volumes:

```
podman volume ls
```

Expected output:

```
DRIVER    VOLUME NAME
```

```
local     website-data
```

Step 4: Inspect the Volume

Display detailed information:

```
podman volume inspect website-data
```

Example output:

```
[
  {
    "Name": "website-data",
    "Driver": "local",
    "Mountpoint": "/home/mausomi/.local/share/containers/storage/volumes/website-
data/_data"
  }
]
```

Notice the **Mountpoint**. This is where Podman stores the data.

Step 5: Create a Container Using the Named Volume

Run an Ubuntu container:

```
podman run -dit \
  --name volume-demo \
  -v website-data:/data \
  docker.io/library/ubuntu:22.04 \
  bash
```

Command Explanation

Option	Description
--------	-------------

-d	Run in detached mode
-i	Keep standard input open
-t	Allocate a terminal
--name	Assign a container name
-v website-data:/data	Mount the named volume inside the container

Step 6: Verify the Container

podman ps

Expected output:

CONTAINER ID	IMAGE	STATUS	NAMES
--------------	-------	--------	-------

xxxxxxxxxxxx	ubuntu:22.04	Up	volume-demo
--------------	--------------	----	-------------

Step 7: Create a File Inside the Volume

Open an interactive shell:

podman exec -it volume-demo bash

Navigate to the mounted volume:

cd /data

Create a file:

echo "Welcome to Podman Named Volumes" > training.txt

Display the file:

cat training.txt

Expected output:

Welcome to Podman Named Volumes

Exit the container:

exit

Step 8: Verify the File Exists

Execute:

```
podman exec volume-demo ls /data
```

Expected output:

```
training.txt
```

Display its contents:

```
podman exec volume-demo cat /data/training.txt
```

Step 9: Stop the Container

```
podman stop volume-demo
```

Step 10: Remove the Container

```
podman rm volume-demo
```

The container has been removed, but the volume still exists.

Step 11: Verify the Volume Still Exists

Run:

```
podman volume ls
```

Expected output:

DRIVER	VOLUME NAME
--------	-------------

local	website-data
-------	--------------

The volume has **not** been deleted.

Step 12: Create a New Container Using the Same Volume

Create another Ubuntu container:

```
podman run -dit \
```

```
--name volume-demo-2 \
```

```
-v website-data:/data \
```

```
docker.io/library/ubuntu:22.04 \
```

```
bash
```

Step 13: Verify the Data Persistence

Display the file:

```
podman exec volume-demo-2 ls /data
```

Expected output:

```
training.txt
```

Display its contents:

```
podman exec volume-demo-2 cat /data/training.txt
```

Expected output:

```
Welcome to Podman Named Volumes
```

This confirms that the data persisted even after the original container was removed.

Step 14: Remove the Second Container

Stop:

```
podman stop volume-demo-2
```

Remove:

```
podman rm volume-demo-2
```

Step 15: Remove the Volume

Delete the volume:

```
podman volume rm website-data
```

Verify:

```
podman volume ls
```

The volume should no longer be listed.

Command Explanations

Command	Description
podman volume create	Creates a named volume
podman volume ls	Lists all volumes
podman volume inspect	Displays volume details
podman run -v	Mounts a volume into a container
podman exec	Executes commands inside a running container
podman stop	Stops a running container
podman rm	Removes a container
podman volume rm	Removes a named volume

Expected Results

After completing this activity, participants should be able to:

- Create and inspect a named volume.
- Mount the volume into a container.
- Store files inside the volume.
- Remove a container without losing data.
- Reuse the volume with another container.
- Remove the volume when it is no longer required.

Activity 9 – Create a Custom Network

Objective

Learn how to create and manage a custom bridge network in Podman. Participants will create a user-defined network, verify its configuration, connect containers to it, and understand how custom networks enable secure communication between containers.

Learning Outcomes

After completing this activity, participants will be able to:

- Create a custom bridge network.
- List available Podman networks.
- Inspect network configuration.
- Attach containers to a custom network.
- Understand the advantages of user-defined networks over the default network.

Estimated Time

20–25 minutes

Prerequisites

- Podman must be installed.

Verify:

```
podman --version
```

- Verify existing networks.

```
podman network ls
```

Example output:

```
NETWORK ID   NAME      DRIVER
xxxxxxxxxx   podman    bridge
```

By default, Podman creates a **bridge network** named **podman**. Any container started without specifying a network is connected to this default bridge.

A **custom bridge network** allows you to:

- Isolate groups of containers.
- Enable container-to-container communication using container names.

- Separate applications running on the same host.
- Customize IP address ranges and subnets.

The general syntax for creating a network is:

```
podman network create NETWORK_NAME
```

Once created, containers can join the network using:

```
podman run --network NETWORK_NAME IMAGE
```

Step-by-Step Instructions

Step 1: List Existing Networks

Run:

```
podman network ls
```

Example output:

NETWORK ID	NAME	DRIVER
xxxxxxxxxx	podman	bridge

Step 2: Create a Custom Network

Create a bridge network named **training-network**.

```
podman network create training-network
```

Expected output:

```
training-network
```

Step 3: Verify the Network

List the available networks again.

```
podman network ls
```

Expected output:

NETWORK ID	NAME	DRIVER
xxxxxxxxxx	podman	bridge
yyyyyyyyyy	training-network	bridge

The new network has been created successfully.

Step 4: Inspect the Network

Display detailed information.

```
podman network inspect training-network
```

Example output :

```
[
  {
    "name": "training-network",
    "driver": "bridge",
    "subnets": [
      {
        "subnet": "10.89.1.0/24",
        "gateway": "10.89.1.1"
      }
    ]
  }
]
```

Notice the:

- Network name
- Driver
- Subnet
- Gateway

Step 5: Create the First Container

Run an Ubuntu container connected to the custom network.

```
podman run -dit \
  --name app1 \
  --network training-network \
```

```
docker.io/library/ubuntu:22.04 \  
bash
```

Step 6: Create the Second Container

Run another Ubuntu container.

```
podman run -dit \  
--name app2 \  
--network training-network \  
docker.io/library/ubuntu:22.04 \  
bash
```

Step 7: Verify the Running Containers

```
podman ps
```

Expected output:

CONTAINER ID	IMAGE	STATUS	NAMES
xxxxxxxxxxxx	ubuntu:22.04	Up	app1
yyyyyyyyyyyy	ubuntu:22.04	Up	app2

Step 8: Verify Network Assignment

Inspect one of the containers.

```
podman inspect app1
```

Locate the **Networks** section.

It should show:

```
training-network
```

Step 9: Display the Network Details

View the containers connected to the network.

```
podman network inspect training-network
```

You should see both containers listed under the network configuration.

Step 10: Stop the Containers

```
podman stop app1 app2
```

Step 11: Remove the Containers

```
podman rm app1 app2
```

Step 12: Remove the Network

Delete the custom network.

```
podman network rm training-network
```

Verify:

```
podman network ls
```

Only the default **podman** network should remain.

Activity 10 – Creating Your First Pod

Objective

Learn how to create a Podman pod and understand how multiple containers can share the same network namespace.

Learning Outcomes

After completing this activity, participants will be able to:

- Create a pod.
- List available pods.
- Inspect pod details.
- Understand how pods group multiple containers.
- Delete a pod.

Estimated Time

20–25 minutes

A **pod** is a group of one or more containers that share resources such as:

- Network namespace
- IP address
- Port mappings
- Loopback interface

Podman provides pods as a first-class feature.

Activities

- Create a pod named web-pod.
- List all pods.
- Inspect the pod.
- Stop the pod.
- Start the pod again.
- Remove the pod.

Commands

```
podman pod create --name web-pod
```

```
podman pod ls
```

```
podman pod inspect web-pod
```

```
podman pod stop web-pod
```

```
podman pod start web-pod
```

```
podman pod rm -f web-pod
```

When you created the pod:

```
podman pod create --name web-pod
```

Podman actually created **one hidden container** called the **infra container**.

So your pod looks like this:

```
web-pod
```

```
|
```

```
└─ Infra Container (created automatically)
```

That's why podman pod ls shows:

```
# OF CONTAINERS
```

```
1
```

even though you haven't added your own application container yet.

What is the Infra Container?

The infra container is a lightweight container whose only purpose is to **hold the shared namespaces** for the pod, such as:

- Network namespace
- IP address
- Hostname
- Shared ports
- IPC namespace (if applicable)

When you later add application containers, they join the namespaces owned by the infra container.

Activity 11 – Deploy Multiple Containers Inside a Pod

Objective

Deploy multiple containers into the same pod and observe resource sharing.

Learning Outcomes

Participants will:

- Deploy containers inside a pod.
- Verify pod membership.
- Understand shared networking.

Activities

Create a pod

```
podman pod create --name app-pod -p 8080:80
```

Deploy Nginx

```
podman run -dt \
```

```
--pod app-pod \
```

```
--name nginx \
```

```
docker.io/library/nginx
```

Deploy BusyBox

```
podman run -dt \
```

```
--pod app-pod \
```

```
--name toolbox \
```

```
docker.io/library/busybox \
```

```
sleep 3600
```

Verify

```
podman ps
```

```
podman pod ps
```

Inspect

```
podman pod inspect app-pod
```

Activity 12 – Managing Pods

Objective

Learn pod lifecycle management.

Learning Outcomes

Participants will:

- Start pods.
- Stop pods.
- Restart pods.
- Remove pods.

Activities

podman pod stop app-pod

Verify

podman ps

Start

podman pod start app-pod

Restart

podman pod restart app-pod

Remove

podman pod rm -f app-pod

Activity 13 – Build Your First Image Using a Dockerfile

Objective

Learn how to build a custom container image using a **Dockerfile**, understand the build context, and run a container from the newly created image using Podman.

Learning Outcomes

After completing this activity, participants will be able to:

- Understand the purpose of a Dockerfile.
- Understand the concept of a build context.
- Build a custom image using Podman.
- Run a container from a custom image.
- Verify the image creation process.
- Inspect the newly created image.

Estimated Time

30–35 minutes

Prerequisites

- Podman must be installed and working.
- Internet connectivity to download the base image.
- Basic knowledge of Linux commands.

Verify the Podman installation:

```
podman --version
```

A **Dockerfile** is a text file containing a series of instructions used to build a container image. Each instruction creates a new image layer, making the image creation process automated, repeatable, and consistent.

During the build process, Podman uses a **build context**, which is the directory specified when running the podman build command. Any files referenced by the COPY or ADD instructions must exist within this build context.

Although this activity uses Podman, the same Dockerfile can also be used with Docker because Podman is fully compatible with Dockerfile syntax.

Step-by-Step Instructions

Step 1: Create a Project Directory

Create a directory that will contain the Dockerfile and application files.

```
mkdir first-image
```

```
cd first-image
```

Verify your current location:

```
pwd
```

Step 2: Create a Website Directory

Create a directory to store the website files.

```
mkdir website
```

Verify:

```
ls
```

Expected output:

```
website
```

Step 3: Create an HTML Page

Create the default web page.

```
nano website/index.html
```

Add the following content:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>Podman Demo</title>
```

```
</head>
```

```
<body>
```

```
  <h1>Welcome to Podman!</h1>
```

```
  <h2>This webpage is served from a custom Docker image.</h2>
```

```
</body>
```

```
</html>
```

Save the file.

Verify:

```
cat website/index.html
```

Step 4: Create a Dockerfile

Create the Dockerfile.

```
nano Dockerfile
```

Add the following content:

```
FROM docker.io/library/nginx:latest
```

```
COPY website/ /usr/share/nginx/html/
```

```
EXPOSE 80
```

```
CMD ["nginx", "-g", "daemon off;"]
```

1. FROM

```
FROM docker.io/library/nginx:latest
```

Purpose

The FROM instruction specifies the **base image** on which your custom image will be built. It is always the first instruction in a Dockerfile (except for comments or parser directives).

In this example:

- **Registry:** docker.io
- **Repository:** library/nginx
- **Tag:** latest

Podman first checks whether this image already exists locally. If it is not available, it downloads it from Docker Hub.

The Nginx image already contains:

- Linux operating system
- Nginx web server

- Required libraries
- Default configuration files

Instead of installing and configuring Nginx manually, you start with a ready-to-use web server.

Image Layer Created: Yes

2. COPY

COPY website/ /usr/share/nginx/html/

Purpose

The COPY instruction copies files from the **build context** (your local project directory) into the container image.

Here:

Source

website/

This refers to the local directory inside the build context.

For example:

first-image/

|

|— Dockerfile

└─ website/

└─ index.html

Destination

/usr/share/nginx/html/

This is the default document root used by Nginx to serve web pages.

After the image is built, the directory inside the image becomes:

/usr/share/nginx/html/

└─ index.html

When someone accesses the web server, Nginx serves this index.html file.

Image Layer Created: Yes

3. EXPOSE

EXPOSE 80

Purpose

The EXPOSE instruction documents that the application inside the container listens on port **80**.

Port 80 is the default HTTP port used by web servers such as Nginx.

It is important to understand that **EXPOSE does not publish the port** to your host machine. It only serves as metadata indicating which port the application expects to use.

To make the application accessible from your host, you still need port mapping when running the container:

```
podman run -d -p 8082:80 podman-demo:v1
```

Here:

- Host Port: **8082**
- Container Port: **80**

Requests to `http://localhost:8082` are forwarded to port 80 inside the container.

Image Layer Created: Yes

4. CMD

```
CMD ["nginx", "-g", "daemon off;"]
```

Purpose

The CMD instruction specifies the **default command** that is executed when a container starts.

Without this instruction, the container would have no default process to execute.

Breaking it down:

```
nginx
```

Starts the Nginx web server.

```
-g
```

Passes a global configuration directive to Nginx.

`daemon off;`

Normally, Nginx runs in the background as a daemon.

Containers are designed to keep running only while their **main process** is active. If Nginx runs in the background, the main process exits immediately, causing the container to stop.

The directive:

`daemon off;`

keeps Nginx running in the foreground, making it the container's main process. As long as this process runs, the container remains running.

Image Layer Created: No

Unlike FROM or COPY, the CMD instruction only defines the default startup command. It does not add files or modify the filesystem.

Save the file.

Verify:

`cat Dockerfile`

Step 5: Understand the Project Structure

Your project directory should now look like this:

first-image/

|

|— Dockerfile

|

└─ website/

└─ index.html

Verify:

`ls -R`

Step 6: Build the Image

Build the image using the current directory as the build context.

```
podman build -t podman-demo:v1 .
```

Notice the **.** (**dot**) at the end of the command.

It specifies that the **current directory** is the build context.

During the build process, Podman performs the following steps:

- Downloads the base Nginx image (if not already available).
- Copies the website directory into the image.
- Creates image layers.
- Tags the image as **podman-demo:v1**.

Step 7: Verify the Image

List all locally available images.

```
podman images
```

Expected output:

REPOSITORY	TAG	IMAGE ID	CREATED
podman-demo	v1	xxxxxxxxxxxx	xx seconds ago

Step 8: Inspect the Image

Display detailed information about the image.

```
podman inspect podman-demo:v1
```

Observe details such as:

- Image ID
- Repository
- Tags
- Layers
- Creation time
- Operating System
- Architecture

Step 9: Run a Container from the Image

Create and start a container.

```
podman run -d \  
--name podman-web \  
-p 8082:80 \  
podman-demo:v1
```

Verify:

```
podman ps
```

Expected output:

CONTAINER ID	IMAGE	STATUS	PORTS
xxxxxxxxxxxx	podman-demo:v1	Up	0.0.0.0:8082->80/tcp

Step 10: Access the Website

Open a web browser and navigate to:

<http://localhost:8082>

The custom webpage should be displayed.

Step 11: View Container Logs

Display the logs generated by the container.

```
podman logs podman-web
```

Step 12: Stop the Container

```
podman stop podman-web
```

Verify:

```
podman ps -a
```

The container status should be **Exited**.

Step 13: Remove the Container

```
podman rm podman-web
```

Verify:

```
podman ps -a
```

The container should no longer be listed.

Command Explanations

Command	Description
podman build	Builds a container image from a Dockerfile.
-t	Assigns a repository name and tag to the image.
.	Specifies the current directory as the build context.
FROM	Specifies the base image.
COPY	Copies files from the build context into the image.
EXPOSE	Documents the port used by the application.
CMD	Specifies the default command executed when the container starts.
podman run	Creates and starts a new container.
podman inspect	Displays detailed information about an image or container.
podman logs	Displays container logs.

Activity 14 – Build a Multi-stage Image Using a Dockerfile

Objective

Learn how to build a multi-stage container image using a Dockerfile and understand how multi-stage builds help create smaller, more efficient production images by separating the build environment from the runtime environment.

Learning Outcomes

After completing this activity, participants will be able to:

- Understand the concept of multi-stage builds.
- Create a Dockerfile with multiple build stages.
- Build an optimized container image using Podman.
- Compare image layers and sizes.
- Understand the benefits of multi-stage builds.

Estimated Time

30–35 minutes

Prerequisites

- Podman must be installed and working.
- Basic knowledge of Dockerfile instructions.

Verify Podman installation:

```
podman --version
```

A **multi-stage build** uses multiple FROM instructions within a single Dockerfile. Each FROM instruction begins a new build stage.

The first stage is typically used to compile or prepare the application, while the final stage contains only the files required to run the application. This approach reduces the final image size by excluding unnecessary build tools, temporary files, and dependencies.

Benefits of multi-stage builds include:

- Smaller image sizes
- Faster image downloads
- Reduced attack surface
- Improved application security

- Better production deployments

Step-by-Step Instructions

Step 1: Create a Project Directory

Create a new project directory.

```
mkdir multi-stage-demo
```

```
cd multi-stage-demo
```

Verify:

```
pwd
```

Step 2: Create an HTML File

Create a simple web page.

```
nano index.html
```

Add the following content:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>Multi-stage Build Demo</title>
```

```
</head>
```

```
<body>
```

```
  <h1>Welcome to the Multi-stage Build Demo</h1>
```

```
  <h2>This webpage was built using a multi-stage Dockerfile.</h2>
```

```
</body>
```

```
</html>
```

Save the file.

Verify:

```
cat index.html
```

Step 3: Create a Dockerfile

Create a Dockerfile.

nano Dockerfile

Add the following content:

Stage 1 - Builder Stage

FROM docker.io/library/alpine:latest AS builder

WORKDIR /build

COPY index.html .

Stage 2 - Runtime Stage

FROM docker.io/library/nginx:latest

COPY --from=builder /build/index.html /usr/share/nginx/html/index.html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]

Understanding the Dockerfile

Unlike the previous activity, this Dockerfile contains **two build stages**. Each stage starts with a FROM instruction.

The first stage prepares the application, while the second stage creates the final runtime image.

Stage 1 – Builder Stage

FROM docker.io/library/alpine:latest AS builder

This instruction starts the first build stage.

It uses the lightweight Alpine Linux image as the base image and assigns the stage the name builder.

Breaking it down:

- **docker.io** → Container registry
- **library/alpine** → Official Alpine Linux image
- **latest** → Image tag

- **AS builder** → Assigns the name builder to this stage

The stage name allows later stages to copy files from this stage.

Image Layer Created: Yes

WORKDIR

WORKDIR /build

The WORKDIR instruction sets the default working directory inside the container.

If the directory does not exist, Podman creates it automatically.

After this instruction, every subsequent command executes inside:

/build

Without WORKDIR, every command would require the full directory path.

Image Layer Created: Yes

COPY

COPY index.html .

Copies the local index.html file into the current working directory.

Because the current working directory is:

/build

the file is copied to:

/build/index.html

Local directory:

multi-stage-demo/

|

├── Dockerfile

└── index.html

Builder stage after the copy:

/build/

└── index.html

Image Layer Created: Yes

Stage 2 – Runtime Stage

FROM docker.io/library/nginx:latest

This instruction begins the second build stage.

Notice something important:

This stage starts with a **fresh Nginx image**.

Everything created in Stage 1 is discarded unless explicitly copied.

This is the key advantage of multi-stage builds.

Instead of carrying the entire builder environment into production, only the required files are copied.

Image Layer Created: Yes

COPY from Previous Stage

COPY --from=builder /build/index.html /usr/share/nginx/html/index.html

This instruction copies a file from the **builder stage** into the final runtime image.

Source Stage

builder

The stage defined earlier.

Source File

/build/index.html

The HTML page created during Stage 1.

Destination

/usr/share/nginx/html/index.html

The default web directory used by Nginx.

Unlike a normal COPY, this instruction copies files **between build stages** rather than from your local computer.

Result inside the final image:

/usr/share/nginx/html/

└─ index.html

Notice that:

- Alpine Linux is **not** included.
- The /build directory is **not** included.
- Only the HTML file is copied.

This keeps the final image clean and lightweight.

Image Layer Created: Yes

EXPOSE

EXPOSE 80

Indicates that the application inside the container listens on port **80**.

This instruction documents the expected network port but does **not** publish it to the host.

To access the application, map a host port when running the container.

```
podman run -d -p 8083:80 multi-stage-demo:v1
```

Host Port:

8083

Container Port:

80

Image Layer Created: Yes

CMD

```
CMD ["nginx", "-g", "daemon off;"]
```

Defines the default command executed when the container starts.

Breaking it down:

nginx

Starts the Nginx web server.

-g

Passes a global configuration directive.

daemon off;

Keeps Nginx running in the foreground.

Containers remain running only while their main process is active. Running Nginx in the foreground ensures the container stays alive.

Image Layer Created: No

How the Multi-stage Build Works

When you execute:

```
podman build -t multi-stage-demo:v1 .
```

Podman performs the following steps:

Stage 1

1. Downloads the Alpine image.
2. Creates the /build directory.
3. Copies index.html into /build.

Stage 2

1. Downloads the Nginx image.
2. Copies only index.html from the builder stage.
3. Places it in:

```
/usr/share/nginx/html/index.html
```

4. Sets the default startup command.

Final image:

Nginx

+

Your HTML Page

Everything else from Stage 1 is discarded.

Save the file.

Verify:

cat Dockerfile

Step 4: Understand the Project Structure

Your directory should look like this:

multi-stage-demo/

|

|— Dockerfile

└─ index.html

Verify:

ls -R

Step 5: Build the Image

Build the image.

podman build -t multi-stage-demo:v1 .

During the build, observe the output.

You should notice:

- Stage 1 begins from Alpine.
- The HTML file is copied into the builder stage.
- Stage 2 begins from Nginx.
- Only the required HTML file is copied into the final image.

Step 6: Verify the Image

podman images

Expected output:

REPOSITORY	TAG	IMAGE ID
multi-stage-demo	v1	xxxxxxxxxxxx

Step 7: View Image History

Display the image history.

```
podman history multi-stage-demo:v1
```

Observe the image layers created from the Dockerfile instructions.

Step 8: Run the Container

```
podman run -d \  
--name multi-stage-web \  
-p 8083:80 \  
multi-stage-demo:v1
```

Verify:

```
podman ps
```

Step 9: Access the Website

Open a web browser and navigate to:

<http://localhost:8083>

The custom webpage should appear.

Step 10: Inspect the Image

Display detailed image information.

```
podman inspect multi-stage-demo:v1
```

Observe:

- Image ID
- Repository
- Tags
- Layers
- Architecture
- Creation time

Step 11: Compare Image Size

Display all local images.

```
podman images
```

Compare the size of:

- podman-demo:v1 (Activity 23)
- multi-stage-demo:v1

Discuss whether the multi-stage image is smaller and why.

Step 12: Stop the Container

```
podman stop multi-stage-web
```

Step 13: Remove the Container

```
podman rm multi-stage-web
```

Verify:

```
podman ps -a
```

Command Explanations

Command	Description
FROM ... AS builder	Creates a named build stage.
WORKDIR	Sets the working directory inside the container.
COPY	Copies files into the current build stage.
COPY --from=builder	Copies files from a previous build stage into the final image.
podman build	Builds a container image from the Dockerfile.
podman history	Displays the history and layers of an image.
podman inspect	Displays detailed image information.
podman run	Creates and starts a new container.

Activity 15 – Build Images with Multiple Tags

Objective

Learn how to assign multiple tags to a container image, understand image versioning, and use tags to manage different versions of the same application.

Learning Outcomes

After completing this activity, participants will be able to:

- Understand the purpose of image tags.
- Create multiple tags for the same image.
- Build images with custom tags.
- Inspect image details.
- Understand image versioning and repository naming conventions.

Estimated Time

20–25 minutes

Prerequisites

- Podman must be installed and working.
- The podman-demo:v1 image should already exist.

Verify the image:

podman images

An **image tag** is a human-readable label assigned to a container image. Tags make it easier to identify different versions of an image without rebuilding it.

For example, a single image can have multiple tags such as:

- v1
- v2
- latest
- development
- production

These tags can all point to the **same image ID**. Creating a new tag does **not** duplicate the image; it simply creates another reference to the existing image.

Image naming convention:

repository:tag

Example:

podman-demo:v1

Where:

- **Repository** = podman-demo
- **Tag** = v1

Step-by-Step Instructions

Step 1: Verify the Existing Image

Display all locally available images.

podman images

Expected output:

REPOSITORY	TAG	IMAGE ID
podman-demo	v1	xxxxxxxxxxxx

Step 2: Create a "latest" Tag

Assign the latest tag to the existing image.

podman tag podman-demo:v1 podman-demo:latest

Step 3: Create a "production" Tag

podman tag podman-demo:v1 podman-demo:production

Step 4: Create a "development" Tag

podman tag podman-demo:v1 podman-demo:development

Step 5: Verify All Tags

List all images.

podman images

Expected output:

REPOSITORY	TAG	IMAGE ID
podman-demo	v1	abc123456789
podman-demo	latest	abc123456789
podman-demo	production	abc123456789
podman-demo	development	abc123456789

Notice that all tags have the **same IMAGE ID**.

Step 6: Inspect an Image

Inspect the latest tag.

```
podman inspect podman-demo:latest
```

Observe:

- Image ID
- Repository
- Tags
- Created Time
- Architecture
- Operating System

Step 7: View Image History

Display the image layers.

```
podman history podman-demo:v1
```

Notice that all tags share the same image history because they reference the same image.

Step 8: Run a Container Using the "latest" Tag

```
podman run -d \  
--name latest-demo \  
-p 8084:80 \
```



```
podman-demo:latest
```

Verify:

```
podman ps
```

Step 9: Verify the Website

Open a web browser and navigate to:

<http://localhost:8084>

The webpage created in Activity 23 should appear.

Step 10: Stop the Container

```
podman stop latest-demo
```

Step 11: Remove the Container

```
podman rm latest-demo
```

Verify:

```
podman ps -a
```

Command Explanations

Command	Description
podman tag	Creates an additional tag for an existing image.
podman images	Lists all local images.
podman inspect	Displays detailed information about an image.
podman history	Displays the image layers and history.
podman run	Creates and starts a new container from an image.

Activity 16 – Build Images Without Using the Build Cache

Objective

Learn how Podman uses the build cache to speed up image creation and understand how to rebuild an image without using cached layers.

Learning Outcomes

After completing this activity, participants will be able to:

- Understand the purpose of the build cache.
- Build images with and without using the cache.
- Observe the difference between cached and uncached builds.
- Identify situations where a clean rebuild is required.
- Compare image build output and history.

Estimated Time

20–25 minutes

Prerequisites

- Podman must be installed and working.
- The first-image project directory should exist.

Navigate to the project directory:

```
cd ~/first-image
```

Verify the project files:

```
ls
```

Expected output:

```
Dockerfile
```

```
website
```

When Podman builds an image, it stores the results of each Dockerfile instruction as an **image layer**. If the Dockerfile and the referenced files have not changed, Podman reuses these existing layers during subsequent builds instead of rebuilding them.

This process is known as **build caching**, and it significantly reduces build time.

However, there are situations where you may want Podman to ignore the cache and execute every instruction again. This is useful when:

- Testing a clean build.
- Verifying dependency updates.
- Troubleshooting build issues.
- Ensuring no outdated cached layers are used.

The `--no-cache` option forces Podman to rebuild every layer from scratch.

Step-by-Step Instructions

Step 1: Build the Image Using the Cache

Build the image normally.

```
podman build -t cache-demo:v1 .
```

Observe the build output.

If you have previously built the image, Podman may reuse cached layers, making the build complete more quickly.

Step 2: Verify the Image

```
podman images
```

Expected output:

REPOSITORY	TAG	IMAGE ID
cache-demo	v1	xxxxxxxxxxxx

Step 3: Modify the Website

Open the HTML file.

```
nano website/index.html
```

Change the content.

Example:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Cache Demo</title>
</head>
<body>
  <h1>Podman Build Cache Demo</h1>
  <h2>This page was rebuilt without using cached layers.</h2>
</body>
</html>
```

Save the file.

Verify the changes.

```
cat website/index.html
```

Step 4: Build the Image Without Using the Cache

Rebuild the image.

```
podman build --no-cache -t cache-demo:v2 .
```

Observe the output carefully.

Unlike the previous build, Podman executes every Dockerfile instruction again instead of reusing cached layers.

Step 5: Verify Both Images

```
podman images
```

Expected output:

REPOSITORY	TAG	IMAGE ID
cache-demo	v1	xxxxxxxxxxxxx
cache-demo	v2	yyyyyyyyyyyyy

Step 6: Compare the Image History

View the history of Version 1.

```
podman history cache-demo:v1
```

View the history of Version 2.

```
podman history cache-demo:v2
```

Compare the output.

Step 7: Run Version 1

```
podman run -d \  
--name cache-v1 \  
-p 8086:80 \  
cache-demo:v1
```

Open:

<http://localhost:8086>

Observe the original webpage.

Step 8: Run Version 2

```
podman run -d \  
--name cache-v2 \  
-p 8087:80 \  
cache-demo:v2
```

Open:

<http://localhost:8087>

Observe the updated webpage.

Step 9: Compare Both Versions

Notice the differences between the webpages running on:

- <http://localhost:8086>
- <http://localhost:8087>

This demonstrates how rebuilding an image after modifying the application produces a new version of the image.

Step 10: Stop the Containers

```
podman stop cache-v1 cache-v2
```

Step 11: Remove the Containers

```
podman rm cache-v1 cache-v2
```

Verify:

```
podman ps -a
```

Command Explanations

Command	Description
podman build	Builds an image from a Dockerfile.
--no-cache	Rebuilds all Dockerfile instructions without using cached layers.
-t	Assigns a repository name and tag to the image.
podman images	Lists locally available images.
podman history	Displays the history and layers of an image.
podman run	Creates and starts a new container.
podman stop	Stops running containers.
podman rm	Removes containers.